

Position: Agent Evaluation Should Be Agentified for Openness, Standardization, and Reproducibility

Xiaoyuan Liu¹ Tianneng Shi¹ Wenbo Guo² Dawn Song¹

Abstract

Agent systems are advancing quickly, but their evaluation remains fragmented: most benchmarks rely on fixed, LLM-centric harnesses that require heavy integration, create test-production mismatch, and limit fair comparison across diverse agent designs. This position paper argues that the root problem is the lack of an open, agent-agnostic assessment interface. We advocate Agentified Agent Assessment (AAA), where evaluation is also performed by an agent and all participants interact through standardized protocols: A2A for task management and MCP for tool access. Conventional benchmarking defines two separate interfaces, one for the benchmark and one for the agent, while AAA only needs one, separating assessment logic from agent implementation and enabling reproducible, interoperable, and multi-agent evaluation. We further introduce AgentBeats as a concrete realization of AAA: it identifies five practical operation modes that make standardized assessment compatible with real-world constraints on openness, privacy, and reproducibility, and provides recommended practices that allow both agent developers and benchmark designers to adopt AAA with minimal additional effort. Together, AAA and AgentBeats offer a clear path toward open, standardized, and reproducible agent assessment.¹

1. Introduction

Large language model (LLM)-based agent systems have seen rapid growth in recent years and are increasingly deployed across diverse tasks. Examples include coding agents (Anthropic; OpenAI, a) that autonomously create

¹UC Berkeley ²UC Santa Barbara. Correspondence to: Xiaoyuan Liu <xiaoyuanliu@berkeley.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹More details are available at <https://agentbeats.org/>.

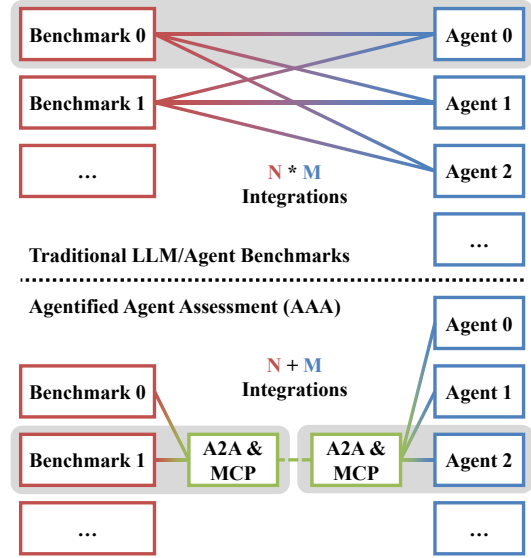


Figure 1. Comparison between Traditional LLM/Agent benchmarks and AAA. AAA reduces the number of integrations from $N \times M$ to $N + M$, while completely separates the benchmark and target agent as shown in the gray box.

and iteratively refine software repositories, browser-based agents (Perplexity AI; OpenAI, b) that retrieve information and execute web actions, and computer-use agents (Sager et al., 2025) that interact directly with graphical interfaces via keyboard and mouse control. This evolution marks a clear shift from standalone foundation models toward agentic systems that combine LLMs with tools, memory, and execution harnesses. Such integration substantially expands the scope of tasks that can be performed and enables capabilities that are difficult or impossible for isolated models.

As a result, a wide range of evaluations and benchmarks have been proposed to measure different aspects of agent capabilities. However, a fundamental compatibility problem quickly emerges. Agent systems differ widely in their architectures and interfaces, while benchmarks impose their own assumptions about input formats, tool APIs, and environment control. An arbitrary agent cannot be directly evaluated on an arbitrary benchmark. In practice, this mismatch leads to a fragile and inefficient evaluation workflow.

Benchmark developers typically integrate a small set of well-known agents through custom, benchmark-specific implementations. This integration effort is often substantial, yet the resulting agent coverage remains limited. As a concrete real-world example, agent implementations such as [OpenHands](#) maintain a dedicated benchmark directory that aggregates dozens of external benchmarks, requiring non-trivial, benchmark-specific adaptation and customization.

The lack of standardization has severe consequences. First, it requires significant effort to align benchmark interfaces with target agents. Even after investing time to learn each benchmark and agent codebase, evaluating M agents on N benchmarks essentially requires $N * M$ bespoke integration efforts as shown in Figure 1. Second, some integrations require changes to both agents and benchmarks. Integrations developed by different parties may also follow different evaluation logic, which may lead to inconsistent or unfair results. Lastly, evaluating a production agent often requires additional customization, creating a test-production mismatch. As a result, reported metrics may fail to capture behaviors or risks that only emerge in real-world deployment.

Standardization is key to resolving this compatibility problem. To enable a benchmark to directly evaluate any agent, the benchmark must make no assumptions about the agent’s internal structure or prior knowledge. However, agents expose diverse interfaces and benchmarks target a wide range of capabilities, which makes it challenging to define a standard that simultaneously (1) is flexible enough to support diverse evaluation scenarios and (2) is simple and lightweight enough to encourage broad adoption.

This position paper advocates two key statements. First, **a universal agent benchmark must be self-contained and present tasks in a way that a human can understand and complete without prior knowledge or external information channels**. Most agents are designed to operate under human instruction, and benchmarks should reflect this interaction pattern by providing clear, self-explanatory instructions and environments. In doing so, benchmarks simulate a human operator issuing commands and judging outcomes, while enabling broader coverage and larger-scale evaluation. It is important to note that this principle does not conflict with presenting tasks in an LLM-friendly format. Specifying required output formats or including few-shot examples remains a recommended practice for high-quality benchmarks. In fact, this aligns naturally with the self-contained task principle: humans also benefit from explicit formats for fair evaluation, and few-shot examples help clarify task requirements.

Second, we argue that **one of the most effective ways to encourage adoption is to reuse existing, widely adopted agent operation standards and repurpose them for evaluation**. Specifically, by leveraging the A2A protocol ([a2a](#),

[2025](#)) for task management and the MCP ([mcp](#), [2025](#)) for tool access, we can cover most agent evaluation scenarios. A2A provides a clear mechanism for task specification, progress tracking, multimodal data exchange, and bidirectional communication. MCP, in contrast, offers structured, self-contained, and format-enforced access to tools and functions. Both protocols are already widely adopted. Most modern agents support A2A natively or can be adapted via a lightweight wrapper using the official A2A libraries, while MCP has become a common practice for environment interaction and external resource access.

Based on these two statements, we further advocate that **agent evaluation itself should be agentified**, since evaluating other agents is a well-defined agentic task. This design brings several advantages.

First, it enables natural adoption of evaluation standards. Framing evaluation as one agent assessing another naturally aligns communication with the agent-to-agent paradigm and fits cleanly within the A2A protocol. Task specifications become standard A2A task messages, and tested agents can interact with environments through MCP tool calls without requiring benchmark-specific harnesses. Second, for evaluations that require human simulation, agentification is a natural solution and provides a clean separation between the simulation-side logic and the tested agent. Third, benchmark design and result judgment become highly flexible. By constructing different evaluator-side agent harnesses, both deterministic workflows and LLM-as-a-judge styles can be supported. Deterministic harnesses can assess task completion using rule-based checks, while generative responses can be evaluated through the same simulated human agents within the benchmark.

From this position, we propose Agentified Agent Assessment (AAA), a new paradigm that treats benchmarks themselves as agents to address the universality challenge in agent evaluation. To show the feasibility of this paradigm, we design AgentBeats, a concrete system that realizes AAA. Within AgentBeats, we expand on implementation and deployment details, as well as potential extensions that further reduce friction for agent development and testing.

The core difference between traditional agent assessment and AAA is the complete separation between benchmarks and the agents being evaluated. In practice, this means that the evaluated agent can reside in a separate repository and be developed by an independent party. This separation relies on two requirements in AAA benchmark design, following the earlier statements. First, the benchmark must make no assumptions about the agent’s internal structure or prior knowledge, assuming only compatibility with A2A and MCP. Second, tasks must be described in a fully self-contained manner through A2A messages, with all necessary context and tools explicitly provided.

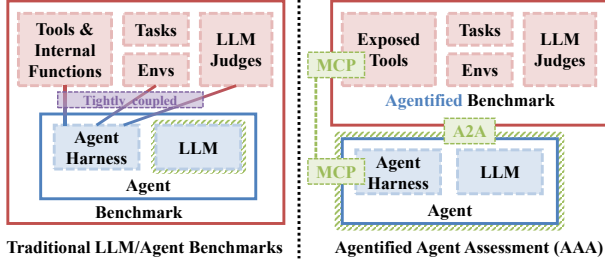


Figure 2. Complete separation between benchmarks and agents. Traditional benchmarks are tightly coupled with fixed agent harness and typically allow only LLM replacement as illustrated by the green shaded area, whereas agentified benchmarks allow the entire agent to be swapped.

As a result, unlike traditional assessments that typically allow only LLM replacement or support a small set of predefined agent harnesses, as shown in Figure 2, AAA enables any A2A- and MCP-compatible agent to be evaluated without additional integration effort. This reduces the integration cost from $N * M$ agent-benchmark combinations to $N + M$ protocol-level integrations as shown in Figure 1.

Beyond efficiency, this standardization also enables openness and reproducibility. An open standard with recommended practices encourages the development of interoperable benchmarks and agents, while an evaluation management platform with multiple operation modes supports reproducible assessments across diverse use cases and deployment settings. Through the design of AgentBeats, we argue that agentifying agent assessment requires minimal additional effort while yielding substantial benefits. Together, AAA and AgentBeats provide a clear path toward open, standardized, and reproducible agent evaluation.

2. AAA Paradigm

Under AAA paradigm, an agent assessment involves three participant roles: a delegator, a judge agent, and one or more subject agents.

- The *judge agent*, acting as an agentified benchmark, implements the evaluation logic and judging procedures, including the test dataset, evaluation workflow, and the metrics to be reported. Despite the name, a judge agent is distinct from an LLM-as-a-judge: the latter, when used at all, is merely one optional scoring component inside a judge agent.
- There may be one or more *subject agents*. A subject agent is the evaluation target that understands A2A tasks and can connect to an MCP server. In multi-agent assessments, multiple A2A-compatible agents are evaluated as separate targets, and their performance metrics are reported independently.

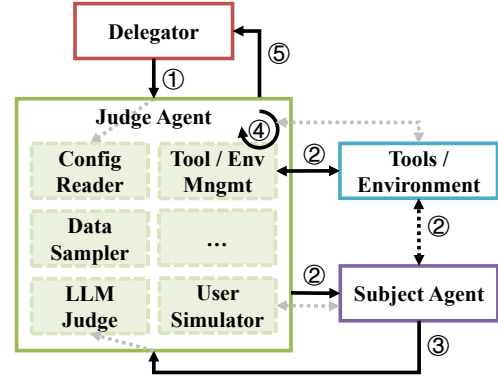


Figure 3. AAA interaction pattern. The internal structure of the judge agent is illustrative rather than prescriptive. Recommended practices for designing judge agents are discussed in Section 4.

- The *delegator* selects a benchmark by choosing a judge agent, describes the evaluation requirements, and specifies the evaluation targets by selecting one or more subject agents.

To better understand these roles, one can imagine a car owner bringing a vehicle to an auto repair shop. The car owner (the delegator) chooses a repair shop or a maintenance package offered by different technicians (judge agents) to inspect the car (subject agent). The technicians performing the examination are responsible for producing the final report.

In practice, the delegator may be a script or a service that issues A2A tasks, while both the judge agent and subject agents must run A2A services. The AAA paradigm itself does not prescribe how roles are mapped to real-world actors or how they are deployed. In Section 3, we describe how this paradigm is instantiated in AgentBeats through five operation modes to accommodate real-world requirements.

Figure 3 illustrates the interaction pattern defined by AAA.

1. **Evaluation request delegation.** The delegator identifies the evaluation goal and configuration, selects the appropriate judge agent and subject agents, specifies the desired result format, and sends a kick-off request to the judge agent via A2A containing this information. AAA does not enforce a specific request format, and we provide a reference format in the discussion of the AgentBeats system.
2. **Task instruction distribution and access authorization.** Based on the request details, the judge agent retrieves relevant data and prepares the evaluation environment, such as launching or configuring an MCP server. It then distributes task instructions, along with environment access information, to the subject agents.

3. **Task fulfillment.** Upon receiving the task instructions, subject agents attempt to complete the task within the allotted time. During execution, depending on task requirements, subject agents may interact with the judge agent one or more times, invoke MCP tools, explore the environment, and take actions.
4. **Performance scoring.** After receiving task completion notifications from subject agents, or when a timeout occurs, the judge agent revokes environment access and inspects both the subject agents' responses and the final environment state to assess performance. Each subject agent is scored according to predefined metrics.
5. **Result reporting.** The judge agent reports evaluation results back to the delegator in the requested format.

This structure naturally extends to multi-agent assessments. In single-agent benchmarking, the judge agent does not need to consider participant roles or capacities, as there is only one evaluation target. In multi-agent assessments, judge agents define, during the design phase, the expected roles and capacities of the subject agents they can assess. Depending on the task type, these specifications may vary. In general, multi-agent tasks can be categorized as collaborative or adversarial.

In adversarial multi-agent tasks, such as chess games (kag, 2025), werewolf-style games (Xu et al., 2024), or simulated CTF competitions (Zou et al., 2026), the roles of subject agents are typically well defined, and specifications can directly follow the team structure or winning conditions of the task. On the other hand, in collaborative multi-agent tasks, the arrangement depends on whether the task explicitly defines a division of labor among agents. Developers may choose either to (1) treat different agents as independent subject agents and score them accordingly, or (2) when the multi-agent structure primarily serves to improve performance while the task itself can be viewed as single-agent, one can organize auxiliary agents as sub-agents and conduct the evaluation as a single-agent assessment.

Beyond the fixed-dataset evaluation pattern implied by the workflow above, AAA naturally supports adaptive assessment. Because the judge agent owns the full evaluation session, the set of tasks need not be enumerated in advance: the judge agent can generate them adaptively, probe weaknesses, or stop early. AAA thus reframes a benchmark from a fixed dataset paired with a scoring function into an interactive evaluation process, concentrating effort on informative cases and avoiding redundant ones. For example, when an agent fails early test cases, the judge agent can skip later ones likely to yield the same result; when an agent does well, it can instead generate harder cases to better differentiate performance.

3. Realizing AAA: AgentBeats

Overview. Applying AAA to real-world scenarios requires more concrete system designs. In practice, one must account for different assessment participants, such as individual developers or organizations, and their roles in a complete evaluation workflow. While the use of A2A and MCP in AAA addresses standardization, three additional dimensions must be considered: openness, privacy, and reproducibility. *Openness* implies encouraging modularized, open-source implementations of both agents and benchmarks, supporting collaborative iteration and publicly accessible assessment services. *Privacy* requires support for closed-source LLMs and agents. In some cases, to ensure fair evaluation and avoid data leakage or overfitting, it is desirable not only to evaluate on hidden data, but also to support private evaluation workflows through closed-source judge agents. *Reproducibility* calls for clear specification and automation of assessment procedures, stability or determinism in judge agent behavior, and explicit state management for subject agents.

To address diverse real-world development and deployment scenarios, we design five operation modes. These modes differ in participant-role mapping, deployment requirements, delegator assignment, and result presentation. For example, some assessments target open-source agent implementations specified as GitHub repositories or Docker images, while others evaluate already deployed, closed-source A2A services. From a system perspective, the agent input to an assessment may take the form of a Git URL, a Docker image reference, or an A2A endpoint URL. We refer to the former cases, where a Git URL or Docker image is provided, as agent blueprints, and to the latter as agent instances. Depending on how agents are provided and by whom, different operation modes apply. To support a complete assessment workflow, each operation mode follows the same three-stage lifecycle, with minor variations in specific steps. We first describe this shared lifecycle, then discuss the design details and differences among operation modes. Finally, we address scalability-related considerations.

3.1. Assessment Lifecycle

Figure 4 illustrates a complete assessment lifecycle with three main stages.



Figure 4. Assessment Lifecycle in AgentBeats. Some steps may be optional in certain operation modes.

Stage I. Agent construction. The goal of this stage is to prepare either an agent blueprint or a live A2A service. Since the delegator’s role is limited to sending A2A messages, which can often be implemented with a few lines of script, we focus primarily on the effort required for agent construction. After implementing the agent, developers may choose to open-source the agent by creating a publicly accessible Git repository. Additionally, by providing a Dockerfile, developers can expose an easily reproducible A2A service. A recommended practice is to make the LLM API key and A2A host configurable through Docker environment variables, further improving reproducibility.

Stage II. Agent registration. This stage gathers all agents relevant to an assessment and optionally makes them publicly accessible. Registration can be as simple as placing agent codebases under a shared directory or specifying agent service URLs in a script. The key objective is to abstract each agent into an easy-to-reference identifier for later stages. In some operation modes, such identifiers are returned as stubs from the registration procedure. When agent blueprints are provided, an optional instantiation step prepares the corresponding live agent instances. The next stage can begin only after all instantiations are complete.

Stage III. Assessment execution. An assigned delegator initiates the evaluation by following the interaction pattern defined in the AAA paradigm. After task completion, results are either returned to the delegator who requested the assessment or made publicly available to support openness.

Next, we further concretize the details of each stage by examining different operation modes. Some steps in certain modes may be optional, simplified, or omitted altogether.

3.2. Operation Modes

We provide five operation modes that all satisfy the AAA paradigm but differ in their real-world use cases in Table 1. A modularized AgentBeats system exposes different system components to support these modes, requiring no or minimal changes to agent implementations while switching between these modes. Below, we describe each operation mode in terms of its suitable scenarios, participant-role mapping, concretized lifecycle, and required system components.

Local Mode. As the simplest and most straightforward realization of the AAA paradigm, local mode requires all participating agents to be instantiated as local services on the same host. This mode is typically suited for local development or fully private, on-prem evaluations. In this setting, all AAA participant roles are mapped to one local developer, with the delegator implemented as a lightweight script.

A common pattern is to implement or download all agent codebases into separate directories during stage I, then write

a script that launches each agent service during stage II. When executing stage III, the script starts or restarts all agent services on predetermined ports, sends the kick-off A2A message, and finally prints the results to the terminal or writes them to a result file. Local mode does not require any dedicated platform components; the only requirement is that all agents conform to the A2A protocol.

Remote Mode. A natural way to orchestrate evaluation and encourage openness is to introduce a centralized platform that manages agents and assessment resources. Remote mode, hosted mode, and proxy mode all adopt this approach and rely on a centralized service component. A platform provider is responsible for deploying and maintaining this backend service and making it accessible to agent developers and assessment users. In remote mode, during stage I, agent providers deploy their own A2A services and make them publicly accessible or reachable by the platform backend. They then submit agent instance information, such as service URLs, to the platform for registration. In stage III, a platform user can request an assessment by selecting registered agents, then monitor execution progress and results through the platform. As implied by its name, agents in remote mode are deployed on remote servers and remain private to other platform users. This mode is well suited for public, collaborative assessments of closed-source agents.

Remote mode relies on a centralized AgentBeats platform that provides agent registration, kick-off message dispatching, and result tracking. It also requires a concrete kick-off request format, since the platform may manage multiple judge agents with different benchmark functionalities. Such a request typically includes three components: a prompt describing the evaluation goal, the URLs of subject agents with role assignments (e.g., black and white in a chess match), and a customizable configuration provided by the requester.

Hosted Mode. Also built on a centralized platform, hosted mode offers a more streamlined and user-friendly experience by allowing agent providers to submit agent blueprints instead of live service URLs. During registration in stage II, an agent provider can submit a GitHub repository that follows a predefined standard (e.g., buildpacks) or a Docker image. The platform then instantiates the agent and verifies that its A2A service is running correctly. Once verified, the agent is marked as ready for subsequent assessments.

Beyond the capabilities required for remote mode, hosted mode requires the platform to handle agent instantiation and service health checks. Instantiation can be implemented using platform-managed clusters or by leveraging third-party container-based serverless infrastructure for improved scalability. As an additional advantage, whereas remote mode typically supports only a single instance per registered agent, hosted mode allows the platform to provision multiple instances to support large-scale evaluations.

Position: Agent Evaluation Should Be Agentified

	Local Mode	Remote Mode	Hosted Mode	Proxy Mode	CI Mode
Participants	One local developer	Remote developer(s) + AB platform	Remote developer(s) + AB platform	One local developer + optional remote developer(s) + AB platform	Remote developer(s) + CI platform
Where are agents instantiated	Local machine	Remote servers	AB platform-managed servers	Local machine + Remote servers	CI platform-managed servers
Who kicks off the assessment	A script on local machine	AB platform	AB platform	AB platform	CI platform (e.g. push triggers)
Where are the results displayed	Local terminal	AB platform	AB platform	AB platform	CI platform (via committed files)
Best suited for	Local development or on-prem assessments	Public assessments of private agents	Public assessments using public agent blueprints	Local development with remote participants	Auditable assessments using public agent blueprints

Table 1. AgentBeats operation modes. The AB platform denotes a centralized platform that orchestrates AAA following the design described in Section 3. The CI platform denotes continuous integration services, such as GitHub Actions.

Proxy Mode. To further support agent development, proxy mode provides an option that combines local development with platform-based evaluation. Consider a developer building a new agent for an upcoming assessment, where other participating agents are already registered on the platform. Remote mode requires full deployment, while hosted mode may incur long build times. Proxy mode addresses this gap by allowing developers to connect a temporary local development server to the platform and reverse-proxy communications between local and remote agents.

From a lifecycle perspective, proxy mode does not require a Git repository or image build. Instead, it assumes an A2A-compatible development server is running locally. In stage II, the local agent connects to the AgentBeats platform to obtain a temporary platform identifier and a message tunnel. By requesting an assessment using this temporary identifier together with other registered agents, and optionally streaming platform-side logs back to the local terminal, developers can test their code against existing agents without leaving their local environment. Supporting proxy mode requires the platform to provide reverse-proxy functionality and expose execution logs through APIs. A platform-provided command-line tool or SDK that automates tunnel management is also important for a smooth developer experience. With proxy mode, judge agent developers can directly test benchmark workflows against well-known agents, while subject agent developers can validate their implementations against agentified benchmarks without downloading datasets or preparing environments.

Continuous Integration (CI) Mode. Reliance on a centralized platform may raise concerns about trust and auditability. Providing an option that avoids a black-box service improves transparency and reproducibility. Motivated by this consideration, CI mode leverages existing public CI infrastructure, such as GitHub Actions, to realize the AAA paradigm without a dedicated platform. This mode can be viewed as migrating local-mode evaluation into CI pipelines.

There are several ways to automate agentified assessments in CI workflows. A simple approach is to commit all files required for local-mode execution into a repository and

configure triggers that invoke the evaluation script on each new commit. In this setup, stage I corresponds to pushing all relevant agent codebases into the repository, along with CI configuration for dependency installation. During stage II, LLM API keys, host addresses, and other environment variables are provided through the CI platform’s secret and token management features to facilitate agent instantiation. Each commit triggers stage III, and results are available in CI logs or committed to a separate branch as artifacts.

Although CI mode does not depend on a centralized platform, it benefits from well-documented reference implementations. Providing example repositories with CI workflows as templates helps establish standard practices. To further encourage modular development, each agent can be wrapped as a subdirectory or submodule with its own Dockerfile or configuration. The CI workflow then orchestrates containerized agent services and executes the assessment in a fully automated and reproducible manner.

4. Developer Recommended Practice

AgentBeats helps subject agent developers test their agents, and also helps benchmark designers standardize evaluation interfaces, reducing integration effort on both sides. In this section, we discuss the implications of practicing AAA and leveraging the AgentBeats system. Specifically, we provide recommended practices for standardizing interfaces while remaining flexible, without introducing excessive additional burden. This section also serves as a guide to help organize the development process for both agents and benchmarks.

4.1. Developing Subject Agents

The basic requirement for subject agent developers is to ensure that their final implementation can accept A2A tasks and connect to external MCP servers specified in the A2A message. In general, there are two cases: (1) the developer already has an agent implementation that does not support these protocols but wants to evaluate it using a judge agent under AAA, and (2) the developer plans to build a new agent from scratch. In the first case, there are three aspects

to check and patch: A2A compatibility, MCP tool access support, and environment adaptivity.

A2A compatibility. If an existing agent does not support A2A (e.g., it exposes a custom API) or only partially supports A2A (e.g., it does not support artifact transfer for media inputs), it is recommended to preserve the original agent package and implement a thin external A2A wrapper that translates task instructions. Since most LLM agents accept natural language instructions in some form before acting, this translation is usually feasible. A concrete example is a coding agent that operates through terminal. In such a case, one can implement a wrapper A2A service that forwards task instructions into a terminal with a clean local working directory, then retrieves output files as artifacts after execution and returns them in the A2A response.

MCP tool access support. While many modern agent frameworks support MCP integration, the key challenge in AgentBeats is that the MCP services used by a subject agent may be provided by the judge agent and specified dynamically in the A2A message at runtime. Supporting dynamic MCP configuration and loading new tool sets often requires additional customization. A simple approach is to handle this within the same thin A2A wrapper layer. In Section 5, we also discuss an alternative approach: providing a default MCP server during the setup phase and dynamically configuring its tool list.

Environment adaptivity. Beyond A2A and MCP, some assessments may require additional capabilities that are natural for humans, thus still being self-contained, but difficult for certain agents. For example, a browser-use task may require navigating to a specific URL, while a terminal-use task may require SSH access to a remote machine and applying changes there. To support such assessments, developers should either implement the required workflows explicitly or incorporate internal tools for the needed capabilities.

In the second case, since AAA only enforces communication protocols, the main considerations largely match those of building a general agent: which framework to use, which LLM to choose, what harness pattern to adopt (e.g., CoT (Wei et al., 2022) or ReAct (Yao et al., 2022)), and whether to incorporate a multi-agent structure, etc. We note that, in practice, it is often effective to optimize against a single judge agent (and potentially a small set of peer subject agents) first, using local mode or proxy mode. This makes development more purpose-driven and efficient. Meanwhile, we encourage developers to make instruction processing as robust as possible. For example, extracting URLs from browser-use instructions is often more reliable when done via an LLM rather than regular expressions, which can improve compatibility across future judge agents that describe similar tasks using different instruction styles.

4.2. Developing Judge Agents

For assessment designers, the first decision is whether to adopt a deterministic workflow to drive evaluation, sometimes even without an LLM, or to incorporate an agentic procedure where parts of the workflow are specified through prompts. For example, incorporating evaluations such as terminal-bench (Merrill et al., 2026) may benefit from a workflow-style design since evaluation is driven by scripts, while incorporating evaluations such as tau-bench (Yao et al., 2025) may benefit more from prompting because user simulation is required. The primary challenge is to design the interaction procedure during assessment execution.

Following the interaction pattern described in Section 2, we recommend organizing the design process around the following questions:

- How is test-related data obtained and sampled?
- How is the evaluation environment prepared and initialized, and how is data loaded?
- How should the task be described, and how should environment access be granted? How should MCP tools be designed and exposed?
- How should the judge agent interact with subject agents throughout assessment?
- During and after task fulfillment, what metrics should be used to score subject agent performance?

Careful consideration of these aspects improves compatibility, supports a wider range of subject agents, and provides a smoother assessment experience. For example:

- It can be useful to make dataset selection configurable through the kick-off request format, so that the requester can control expected runtime and scope.
- Avoiding repeated environment rebuilds by supporting reset semantics, and reducing unnecessary data transmission, can improve evaluation efficiency.
- Choosing appropriate tasks and scoring granularity can help subject agents better understand objectives, while clear tool descriptions help improve performance.
- Using a sub-agent inside the judge agent can help isolate scoring logic from simulated user conversations, reducing the risk of prompt-injection-based cheating.
- Including metadata such as sampled data point identifiers and dataset size enables aggregation across runs.

Note that beyond building judge agents for new scenarios and datasets, the process of agentifying existing benchmarks follows the same design procedure. The key challenge is to map the benchmark’s data and evaluation logic onto the corresponding components discussed above.

5. Extension and Integrations

The AAA paradigm also makes it easier to introduce additional abstractions that improve agent development and evaluation utilities. In this section, we discuss potential extensions to the AgentBeats implementation, as well as possible integrations. Some extensions are necessary or particularly useful for enabling features in certain operation modes. Others focus on improving openness and collaboration, or emphasize observability, providing greater transparency and assessment insights that support agent development and iteration.

Agent Control Plane. Agent blueprints in hosted mode provide transparency at the cost of privacy and deployment overhead, while agent instances in remote mode preserve privacy at the cost of reduced visibility. For example, when a remote agent instance exhibits errors or poor performance, execution logs are often unavailable. Similarly, when switching to a new assessment, there is typically no standardized mechanism to reset a remote agent service. To bridge this gap, an agent control plane serves both as an operation extension and an observability layer. It is implemented as a thin service wrapped around the A2A service, providing APIs that allow a centralized platform to restart agent process and stream execution logs back to the platform.

Agent and Tool Authentication. Concerns about abuse of publicly exposed agent services can be addressed through additional authentication components for agents and tools. Such functionality is essential for building a practical, production-ready platform that supports controlled access.

MCP Gateway. To address the dynamic MCP configuration challenge discussed earlier, one solution is to introduce an MCP gateway service with a fixed endpoint that supports dynamic control over tool access. As an operation extension, a judge agent can connect its dynamically created MCP server as a provider to this gateway and selectively expose tools to authenticated subject agents. In this design, as long as a subject agent connects to the gateway using credentials tied to its agent identifier, dynamic MCP reconfiguration is handled transparently and no longer needs to be implemented inside the subject agent.

Sandbox and Container Manager. A shared sandbox and container management service can be introduced to assist judge agents. This service centrally manages computational resources and avoids complex docker-in-docker setups when judge agents are themselves containerized. Additionally, the manager may expose an MCP interface, making it easier to design judge agents using prompt-driven workflows that require dynamic environment control.

Agent Registry. An agent registry is a collaboration-focused extension that supports discovery of both subject agents and benchmarks that are represented as judge agents.

It maintains a publicly accessible list of platform identifiers and associated metadata, potentially derived from A2A agent cards, enabling users to efficiently select appropriate judge agents and subject agents for a given assessment.

Agent Leaderboard. To provide clearer visibility into agent performance and support ranking among multiple subject agents evaluated under the same judge agent, an agent leaderboard can be introduced. To improve interoperability, the leaderboard may also define recommended metric reporting formats, simplifying judge agent response parsing and result aggregation.

Beyond extending AgentBeats directly, the AAA paradigm is also integration-friendly and can interoperate with a range of third-party services to improve overall utility. For example, it can integrate with [Hugging Face](#) for richer leaderboard functionality, [LiteLLM](#) and [MLflow](#) for improved LLM access observability, and [Weights & Biases](#) for logging and experiment management. AAA may also facilitate trajectory collection for reinforcement learning. To bridge the gap between Gymnasium-style APIs commonly used in RL and the protocol-based interaction in AAA, AgentBeats supports translation from type-specified Gym interfaces to self-descriptive MCP tools, enabling any MCP-compatible agent to explore RL environments without native Gym support. Combined with environment-management systems such as OpenEnv ([Meta](#)), AAA provides a universal interaction standard, while also introducing an intermediate layer that facilitates tracking and analysis of agent behavior across training and evaluation.

Concurrent with AgentBeats, CUBE ([Lacoste et al., 2026](#)) standardizes how individual benchmark environments are packaged, deployed, and lifecycle-managed, independently of any evaluation framework. The two standards are naturally composable: a judge agent can use a CUBE-compliant benchmark as its internal infrastructure substrate, distributing the resulting MCP tool endpoints via A2A while AAA orchestrates the overall evaluation campaign.

6. Alternative Views

While we advocate agentifying agent assessment, other approaches may also help mitigate compatibility challenges.

A brute-force approach is to continually expand the number of task-specific agent harnesses until broad coverage is achieved. NeMo ([NVIDIA](#)) follows this direction by providing dozens of different harnesses for diverse tasks within a single repository. The advantage of this approach is that evaluation types are carefully curated and manually aligned. However, it requires substantial engineering effort to achieve and maintain high coverage, is difficult to scale through community adoption, and lacks flexibility in supporting multiple harness designs for the same task.

Another approach is to leverage shared standards other than A2A. Harbor (Harbor Framework Team, 2026), for example, enforces a terminal-based interaction interface combined with checking scripts. Its clear structure and expressive power make it a strong candidate for standardization. However, unlike A2A, which is designed specifically for agent communication, terminal-based serialization may be less suitable for UI-centric tasks such as browser interaction.

A third option is to introduce an entirely new standard. HAL (Kapoor et al., 2025) follows this path by defining a minimal agent interface designed to reduce learning and integration costs. Exgentic (Bandel et al., 2026) pursues a similar direction with adaptors that leave agent-side interfaces untouched, though benchmarks must still be restructured to fit its protocol. By retaining only abstractions necessary for evaluation, such systems can remain efficient and clean. However, promoting adoption of a new standard remains challenging due to the required manual adaptation effort. Moreover, unlike A2A, which directly targets production use, interfaces designed specifically for evaluation risk introducing test-production mismatch.

7. Conclusion

Agent evaluation today suffers from severe compatibility issues due to the lack of standardization, while limited openness and reproducibility further hinder progress. This position paper argues that agentifying the agent assessment is a necessary step forward, and provides a practical roadmap to fundamentally improve agent benchmarking practice.

Acknowledgements

We thank Chenguang Wang, Daniel Miao, Donghyun Lee, Evan Sandoval, Gal Gantar, Jingxuan He, Jinhao Zhu, Sihan Ren, Siyuan Xie, Yiyun Sun, Zhe Ye, and Zhun Wang for their help with this work. We also thank the RDI Foundation for their support and helpful discussions.

References

- Agent2agent (a2a) protocol specification. Online draft specification, 2025. URL <https://a2a-protocol.org/latest/specification/>. Draft v0.3.0.
- Kaggle game arena, 2025. URL <https://www.kaggle.com/game-arena>. A benchmarking platform where AI models and agents compete in strategic games.
- Model context protocol (mcp) specification. Online technical specification, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25>.
- Anthropic. Claude code. URL <https://code.claude.com/docs/en/overview>.
- Bandel, E., Yehudai, A., Eden, L., Sagron, Y., Perlitz, Y., Venezian, E., Razinkov, N., Ergas, N., Ifergan, S. S., Shlomov, S., Jacovi, M., Choshen, L., Ein-Dor, L., Katz, Y., and Shmueli-Scheuer, M. General agent evaluation, 2026. URL <https://arxiv.org/abs/2602.22953>.
- Harbor Framework Team. Harbor framework: A framework for evaluating and optimizing agents and models in container environments. <https://github.com/lau-de-institute/harbor>, 2026.
- Hugging Face. Hugging face. <https://huggingface.co>. Open platform for machine learning models, datasets, and evaluation.
- Kapoor, S., Stroebel, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., Xue, T., Chen, Z., Chen, F., Utpala, S., Ndzomga, F., Oruganty, D., Luskin, S., Liu, K., Yu, B., Arora, A., Hahm, D., Trivedi, H., Sun, H., Lee, J., Jin, T., Mai, Y., Zhou, Y., Zhu, Y., Bommasani, R., Kang, D., Song, D., Henderson, P., Su, Y., Liang, P., and Narayanan, A. Holistic agent leaderboard: The missing infrastructure for ai agent evaluation. <https://github.com/princeton-plti/hal-harness>, 2025.
- Lacoste, A., Gontier, N., Shliazhko, O., Jaiswal, A., Sareen, K., Nanisetty, S., Cabezas, J., Verme, M. D., Younis, O. G., Baratta, S., Avalle, M., Kerboua, I., Lù, X. H., Bandel, E., Shmueli-Scheuer, M., Yehudai, A., Choshen, L., Lebensold, J., Hughes, S., Caccia, M., Drouin, A., Reddy, S., Yu, T., Su, Y., Neubig, G., and Song, D. Cube: A standard for unifying agent benchmarks, 2026. URL <https://arxiv.org/abs/2603.15798>.
- LiteLLM. Litellm. <https://github.com/BerriAI/litellm>.
- Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J. Y., Walshe, T., Buchanan, E. K., et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Meta. Openenv. <https://github.com/meta-pytorch/OpenEnv>. Open-source platform for environment-based agent training and evaluation.
- MLflow. MLflow. <https://github.com/mlflow/mlflow>.
- NVIDIA. Nemo evaluator. <https://github.com/NVIDIA-NeMo/Evaluator>.

OpenAI. Openai codex, a. URL <https://openai.com/codex/>.

OpenAI. Chatgpt atlas, b. URL <https://chatgpt.com/atlas>.

OpenHands. Openhands repository. <https://github.com/OpenHands/OpenHands/tree/main/evaluation/benchmarks>.

Perplexity AI. Comet. URL <https://www.perplexity.ai/comet>.

Sager, P. J., Meyer, B., Yan, P., von Wartburg-Kottler, R., Etaiwi, L., Enayati, A., Nobel, G., Abdulkadir, A., Grewe, B. F., and Stadelmann, T. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *arXiv preprint arXiv:2501.16150*, 2025.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Weights & Biases. Weights & biases. <https://wandb.ai>.

Xu, Z., Yu, C., Fang, F., Wang, Y., and Wu, Y. Language agents with reinforcement learning for strategic play in the werewolf game. In *International Conference on Machine Learning*, 2024.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025.

Zou, Y., Liu, J., and Fan, W. Ctfagent: An llm-powered agent for ctf challenge solving. *Journal of Information Security and Applications*, 96:104305, 2026.